

MANUEL WINDOWS

Manuel d'utilisation — CodeScan sur Windows

Ce manuel explique comment utiliser **CodeScan** (analyseur statique de code, **sans API d'IA**) sur **Windows**, pour auditer vos projets locaux, générer des rapports HTML/JSON, comprendre la note /100 et utiliser le mode « IRON MAN AI » (audit web Kali) si vous avez des outils de sécurité dans le PATH.

CodeScan est un outil de **défense** : analysez vos propres projets ou ceux que vous êtes autorisé à auditer.

1. Installation

1. Installez **Python 3.10+** depuis python.org (cochez « Add Python to PATH » lors de l'installation).
1. Décompressez le dossier `codescan` où vous voulez.
1. (Optionnel) Installez `requests` uniquement si vous voulez la vérification des CVE de dépendances via OSV.dev :

```
pip install -r requirements.txt
```

Le cœur de CodeScan n'utilise **que la bibliothèque standard** : aucune dépendance obligatoire.

2. Premier scan statique

Ouvrez PowerShell (ou Git Bash) dans le dossier `codescan` et lancez :

```
uv run python main.py --path .\examples --output rapport.html
```

Vous obtenez dans le terminal le **résumé console** et, dans `rapport.html`, la **note /100** (avec sa lettre A→F), des cartes de synthèse, les findings **groupés par niveau** (CRITIQUE / À REVOIR / MINEUR) et la répartition par catégorie.

Comprendre la note /100

Élément	Détail
Formule	$100 / (1 + \text{densité}/K) \times (1 - 0,5 \times \text{proportion de critiques})$
Pondérations	critical=5, high=2, medium=1, low=0.5
Lettres	A≥90, B≥80, C≥70, D≥40, F<40
Domaines	Sécurité, Qualité, Performance

Plus il y a de résultats graves, plus la note baisse. Le score est calculé sur **l'ensemble des résultats dédoublés**, indépendamment du filtre

```
--severity-min .
```

3. Options principales

```
# Analyser un dossier local
uv run python main.py --path C:\mon\projet

# Analyser un dépôt GitHub (cloné puis nettoyé)
uv run python main.py --repo https://github.com/user/repo

# Rapport JSON structuré, filtré aux failles haute/critique
uv run python main.py --path C:\mon\projet --output scan.json --severity-min=high

# Sans qualité ni score (pour un audit sécurité pure)
uv run python main.py --path C:\mon\projet --no-quality --no-score --output scan.html
```

Option	Effet
<code>--path <dossier></code>	Projet local à analyser
<code>--repo <URL></code>	Dépôt GitHub à cloner puis analyser
<code>-o, --output <fichier></code>	Rapport <code>.json</code> ou <code>.html</code>
<code>--severity-min</code> <code>{critical,high,medium,low}</code>	Filtre le listing (défaut : low)
<code>--no-deps</code>	Désactive OSV.dev (CVE de dépendances)
<code>--no-quality</code>	Désactive les métriques de qualité
<code>--no-score</code>	Ne calcule ni n'affiche la note /100
<code>--exclude <motif></code>	Exclut les fichiers dont le chemin contient le motif (répétable, jokers <code>*</code> / <code>?</code>)
<code>-v, --verbose</code>	Sortie détaillée
<code>--version</code>	Version

Code de sortie : `0` si aucun résultat critique/haute, `1` sinon

(utile pour un CI).

4. Imprimer un rapport en PDF (sans dépendance)

Le rapport HTML est **autonome** (CSS embarqué, aucun accès réseau) et optimisé pour l'impression A4.

1. Ouvrez `rapport.html` dans **Edge, Chrome ou Firefox**.
1. **Ctrl+P** → choisissez « **Enregistrer au format PDF** » (destination PDF).
1. Vérifiez la mise en page : format **A4**, marges **12 mm**, les cartes et relevés **ne sont pas coupés** (`break-inside: avoid`).

Les manuels (ce document) peuvent aussi être convertis en PDF via

`docs\make_pdf.py` (détection automatique d'Edge/Chrome/Chromium en

headless), ou `docs\make_pdf_fpdf2.py` avec `pip install fpdf2`.

5. Mode IRON MAN AI — audit web (Kali, utilisable sous Windows)

L'audit web s'appelle **IRON MAN AI** (`kali_scan.py` / `ironman.py`).

Il est conçu pour **Kali Linux** ; sous Windows il fonctionne de la même

façon si les outils sont dans le PATH (nmap, nikto, etc.), sinon le

préflight vous le dira et le scan s'arrêtera (ou continuera avec

`--allow-missing`).

```
# Vérifier les outils disponibles
uv run python kali_scan.py --check

# LA commande unique : tout l'audit, au maximum, sans limite de temps,
# avec l'audit complet en PDF (JSON + HTML + PDF)
uv run python ironman.py --url http://127.0.0.1:8000 --authorized

# Simuler (affiche les commandes sans rien exécuter)
uv run python kali_scan.py --url http://127.0.0.1:8000 --authorized --dry-run

# Scan raisonnable d'un serveur local vulnérable de démonstration
uv run python examples\vuln_server.py --port 8123
uv run python kali_scan.py --url http://127.0.0.1:8123 --authorized --allow-missing
```

L'audit web est **non invasif par défaut** ; le flag `--attack` (sqlmap,

xsstrike, commix, hydra) est réservé aux cibles explicitement autorisées

(`--authorized` est obligatoire). Le PDF de l'audit complet est généré

100 % stdlib (aucun navigateur requis).

6. Dépannage Windows

Problème	Cause	Solution
python introuvable	Python absent du PATH	Réinstaller en cochant « Add to PATH »
Caractères accentués illisibles	Encodage console	Le code utilise UTF-8 ; sous vieux terminal, utiliser Windows Terminal
Use uv run python	uv actif	Taper uv run python ... (ou désactiver uv local)
--repo échoue	Git introuvable	Installer Git for Windows
Scan web vide	Outils Kali absents	Lire kali_scan.py --check et installer les outils

7. En résumé

1. `uv run python main.py --path <projet>` pour un **audit statique** ;
1. ouvrez le HTML → **Ctrl+P** → **Enregistrer en PDF** ;
1. `uv run python ironman.py --url <cible> --authorized` pour l'**audit web complet en PDF** (sur Kali, après le `--check` et l'installation des outils) ; `kali_scan.py --url <cible> --authorized` pour le scan web « raisonnable ».

Rappel : IRON MAN AI (CodeScan) est un outil de **défense**, destiné à vos projets ou à ceux que vous êtes autorisé à auditer.

Généré par CodeScan — manuel imprimable.